

Android Abstract Namespace UNIX Domain Socket IPC & SCM_RIGHTS Specification

1. Architectural Rationale: Abstract Linux Namespace

Traditional UNIX domain sockets create a filesystem node (e.g. `/tmp/aesc_shell.sock`). On Android, this approach introduces security and performance bottlenecks:

1. **Filesystem Permission Barriers:** Android app sandboxing strictly limits writing to shared filesystem paths across different UID domains.
2. **Flash Wear & Disk I/O Latency:** Creating and unlinking socket files introduces physical storage overhead.

The **Linux Abstract Socket Namespace** solves both issues. By prefixing the socket path with a null byte (`\0`), the socket address is maintained entirely in kernel memory without creating a node on the physical filesystem:

- Disappears automatically when all referencing processes terminate.
- Eliminates filesystem permission collisions between distinct daemon UIDs.
- Zero disk I/O latency.

2. Server Implementation: Abstract Socket Binding

```
#include <sys/socket.h>

#include <sys/un.h>

#include <unistd.h>

#include <string.h>

int create_abstract_server_socket(const char* name) {

    int server_fd = socket(AF_UNIX, SOCK_STREAM, 0);

    if (server_fd < 0) return -1;

    struct sockaddr_un addr;
```

```

memset(&addr, 0, sizeof(addr));

addr.sun_family = AF_UNIX;

// First character MUST be null byte '\0' to target abstract namespace

addr.sun_path[0] = '\0';

strncpy(&addr.sun_path[1], name, sizeof(addr.sun_path) - 2);

// Calculate length including abstract null prefix and string payload

socklen_t len = sizeof(addr.sun_family) + 1 + strlen(name);

if (bind(server_fd, (struct sockaddr*)&addr, len) < 0) {

    close(server_fd);

    return -1;

}

if (listen(server_fd, 8) < 0) {

    close(server_fd);

    return -1;

}

return server_fd;

}

```

3. Inter-Process File Descriptor Passing via SCM_RIGHTS

To transfer an `ASharedMemory` file descriptor from `Aesc` (or `Horizons UI`) to the NPU engine daemon (`:qairt_engine`):

Sender Implementation (**sendmsg**)

```
int send_shared_fd(int socket_fd, int fd_to_send) {  
  
    struct msghdr msg = {0};  
  
    char buf[CMSG_SPACE(sizeof(int))];  
  
    memset(buf, 0, sizeof(buf));  
  
    struct iovec io = { .iov_base = (void*)"FD", .iov_len = 2 };  
  
    msg.msg_iov = &io;  
  
    msg.msg_iovlen = 1;  
  
    msg.msg_control = buf;  
  
    msg.msg_controllen = sizeof(buf);  
  
    struct cmsghdr *cmsg = CMSG_FIRSTHDR(&msg);  
  
    cmsg->cmsg_level = SOL_SOCKET;  
  
    cmsg->cmsg_type = SCM_RIGHTS;  
  
    cmsg->cmsg_len = CMSG_LEN(sizeof(int));  
  
    memcpy(CMSG_DATA(cmsg), &fd_to_send, sizeof(int));  
  
    return sendmsg(socket_fd, &msg, 0);  
  
}
```

Receiver Implementation (**recvmsg**)

```
int receive_shared_fd(int socket_fd) {  
  
    struct msghdr msg = {0};  
  
    char buf[CMSG_SPACE(sizeof(int))];  
  
    char dummy[2];  
  
    ...  
}
```

```

struct iovec io = { .iov_base = dummy, .iov_len = sizeof(dummy) };

msg.msg_iov = &io;

msg.msg_iovlen = 1;

msg.msg_control = buf;

msg.msg_controllen = sizeof(buf);

if (recvmsg(socket_fd, &msg, 0) < 0) return -1;

struct cmsghdr *cmsg = CMSG_FIRSTHDR(&msg);

if (!cmsg || cmsg->cmsg_level != SOL_SOCKET || cmsg->cmsg_type != SCM_RIGHTS) {

    return -1;

}

int received_fd;

memcpy(&received_fd, CMSG_DATA(cmsg), sizeof(int));

return received_fd;

}

```

4. Integration with Android Manifest Decoupled Daemons

In `AndroidManifest.xml`, daemon processes are configured to listen on designated abstract endpoints:

- `:orchestrator_daemon` → `\0aesc_orchestrator`
- `:qairt_engine` → `\0qairt_engine_ipc`
- `:llamacpp_engine` → `\0llamacpp_engine_ipc`
- `:media_daemon` → `\0aeyre_media_stream`